

基于 DataCyber 的 Data Agent: 构建企业数据智能中枢的工程范式

演讲人：许锡彬

数新智能 总架构师

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

- 01 为什么需要数据智能中枢
- 02 SRW——把专家经验变成可执行系统
- 03 GPME——从协同走向可委派自主
- 04 实践价值与落地路径
- 05 DataCyber的Data Agent Ready介绍
- 06 Q&A

01 为什么需要数据智能中枢

模型会推理，不等于企业数据能力会行动

Agent 时代，数据系统要从“供人使用”转向“供 Agent 使用”

01 可理解

Agent 能识别业务实体、指标、维度与统一口径。

02 可访问

通过标准工具和最小权限，安全获取与操作数据。

03 可追溯

每个结论、产物和动作，都能还原来源与过程。

04 可治理

权限、质量、规范与审计直接进入执行链路。



本场聚焦：数据供给与治理中枢 + 数据研发闭环 + 从阶段 2 到阶段 3 的工程路径

企业不是缺数据，也不是缺模型，而是二者没有形成闭环



模型能力持续提升 ↑



模型能力持续提升，并不会自动填平企业语义、权限与流程的缺口。

阶段 3 不是“更长的对话”，而是人机责任边界的变化

01

AI 辅助人

形态	Copilot + IDE
人的职责	拆任务、操作、审结果
AI 职责	SQL / 代码局部生成
产品单元	一次生成
参考效率	约 +30%

02

人机协同

形态	Data Agent 1.0
人的职责	定步骤、补上下文、设检查点
Agent	调工具、串联任务
产品单元	会话 / 任务
参考效率	约 +30% ~ 100%

03

可委派自主

形态	云端托管 Data Agent
人的职责	定 Goal、规则与验收
Agent	规划、执行、监控、自愈
产品单元	一个 Goal
目标效率	10X+, 逐级解锁

人：操作员
拆任务 · 操作系统 · 审结果

人：协作者
定步骤 · 补上下文 · 设检查点

人：目标与例外管理者
目标定义 · 规则制定 · 验收与例外决策

如果没有目标验收、风险策略和评测闭环，“自主”只是更长、更不可控的 ReAct。



数据智能中枢 = 可调用底座 + 语义契约 + 可信 Agent 闭环

MCP: 能力调用

A2A: 任务委派

Data Agent 层	数据开发 需求 · 任务 · 代码	数据治理 语义 · 质量 · 血缘	数据分析 问数 · 归因 · 行动	运维诊断 根因 · 资源 · 优化
可信控制面	GPME 目标 · 策略 · 记忆 · 评测	Context Plane 检索 · 裁剪 · 隔离	身份 · 审批 密钥 · 审计	Trace · Budget Run State
Agent Runtime	Skill 能力契约	Role 责任边界	Workflow 执行闭环	Durable Execution 恢复 · 幂等 · 补偿
DataCyber 数据面	多云数据集成	湖仓 · 流批 · 多引擎	资产 · 质量 · 数据服务	语义契约 · 可信资产

中枢不是新的数据平台，而是让已有数据能力变成 Agent 可理解、可调用、可治理的运行体系。

02 SRW—把专家经验变成可执行系统

让能力可复用、责任可治理、交付可闭环

“能调用工具”与“能稳定交付”之间，还有四道鸿沟

1

不稳定

相同需求因提示词、上下文或调用顺序不同，产生不同结果。

2

难使用

使用者仍需知道选哪个工具、如何构造参数、怎样判断结果。

3

无闭环

工具是离散能力，需求、探查、开发、测试与发布仍靠人工串联。

4

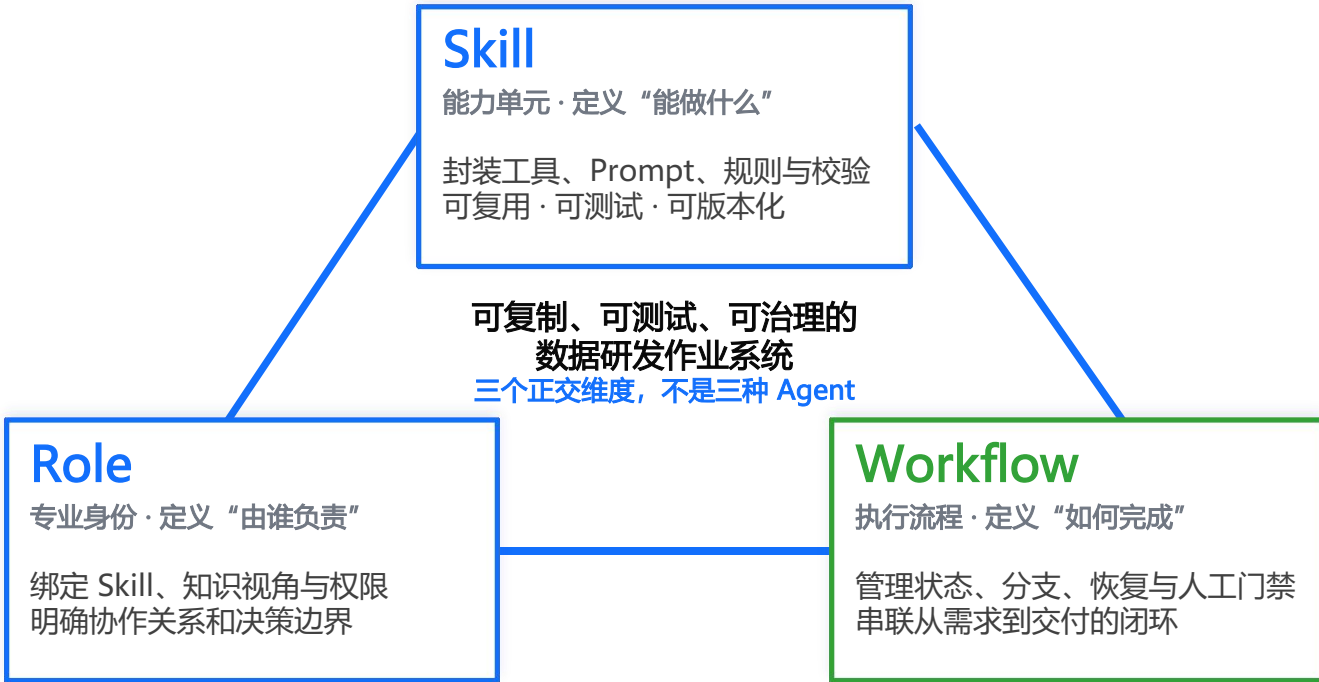
无语义

模型不理解业务实体、指标口径和建模规范，结果能跑但未必正确。

企业需要的不是更大的工具箱，而是一个懂业务、有规范、能闭环的专属 Data Engineer。

SRW · EXECUTION TRIANGLE

把分散工具重组为有能力、有身份、有流程的作业系统



工程对象与实践刻度

同类数据研发项目的参考实践，把 SRW 从概念落到了可管理对象：

9 自建业务 Tools	200+ 平台系统工具 · MCP
20+ 原子 Skills	8 专业 Roles

10 类场景
先分流，再进入研发主链或专属路径

数字用于说明 SRW 的工程化粒度，不代表 DataCyber 必须复制相同数量。

Skill 让能力可复用	Role 让责任可治理	Workflow 让交付可闭环
--------------	-------------	-----------------

Role 是职责抽象，不等于一个独立 Agent

默认单 Agent；只有上下文隔离、权限隔离或独立扩缩容产生明确收益，并通过可重复 Eval，才解锁多 Agent。

Skill 的完成标准不是“模型说做完了”，而是产物通过契约



生产级 Skill 应具备的 9 项能力 不是普通 Prompt 模板

能力描述

适用与不适用场景

输入契约

结构化 Schema

上下文契约

语义、状态与预算

执行资源

工具、脚本、模板与知识

规范校验

命名、分层、敏感字段

输出契约

产物、证据、风险与状态

评测用例

正常、边界、失败与回归

副作用声明

只读、草稿、生产变更

运行语义

幂等、超时、重试与补偿

Skill 标准包结构

```

<skill_id>/
├── SKILL.md # 步骤、约束、模板 + YAML
├── prompts/ # 提示词模板
├── tools/   # 自定义工具实现
├── tests/   # 测试用例与数据
└── meta.yaml # 元数据索引
    
```

契约通过 + 安全可执行 + 上下文高效 + 工具调用评测通过

DURABLE WORKFLOW

Role 管责任, Workflow 管持久状态, MCP / A2A 管连接与协作

数据开发场景示例 以下七阶段 Workflow 与 8 个 Role 是数据开发领域的职责映射；SRW 范式本身可复用于治理、运维、分析等其他场景。

8 个专业 Role 每个 Role 绑定不同的 Skill 白名单与知识视角；**同一个 Skill 可以被多个 Role 复用**。



SRW · ROLE IMPLEMENTATION

Role = Profile: 绑定 Skill、工具面、知识视角与权限边界

Spike / 快速验证示意 · 编排内核建议 LangGraph (或同等状态图) · 非 DataCyber 生产承诺

① 静态配置 / Registry

YAML · JSON · Pydantic: skills / tool_allowlist / knowledge_view / authz_scope / handoff_contract。LangGraph 没有内置 Role; 这是业务侧定义。

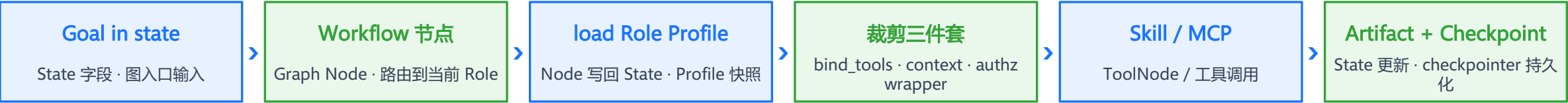
② 放入 Graph State

节点 load_role 按 goal / 当前节点写入 state.active_role + state.role_profile (快照)。进 Checkpoint, 可回放、可审计。

③ 生效: 被节点逻辑消费

后续 node 读 Profile: bind_tools、上下文裁剪、Tool wrapper 鉴权、handoff 校验。配置不自动生效, 必须被代码消费。

字段	一句话
role_id	当前节点由谁负责 → 写入 state.active_role
skills[]	可挂载的能力契约
tool_allowlist	可见/可调 MCP 工具 (含副作用等级) → bind_tools
knowledge_view	本步允许注入的语义 / 规范 / 样例 / Memory 范围
authz_scope	用户 · Agent · 环境 · 资源的最小权限包 → 执行前二次闸
handoff_contract	离开本节点必须落盘的结构化 Artifact → 条件边守卫



先验证「带权限的职责配置进 State 并被节点消费」, 再验证「要不要拆多 Agent」。

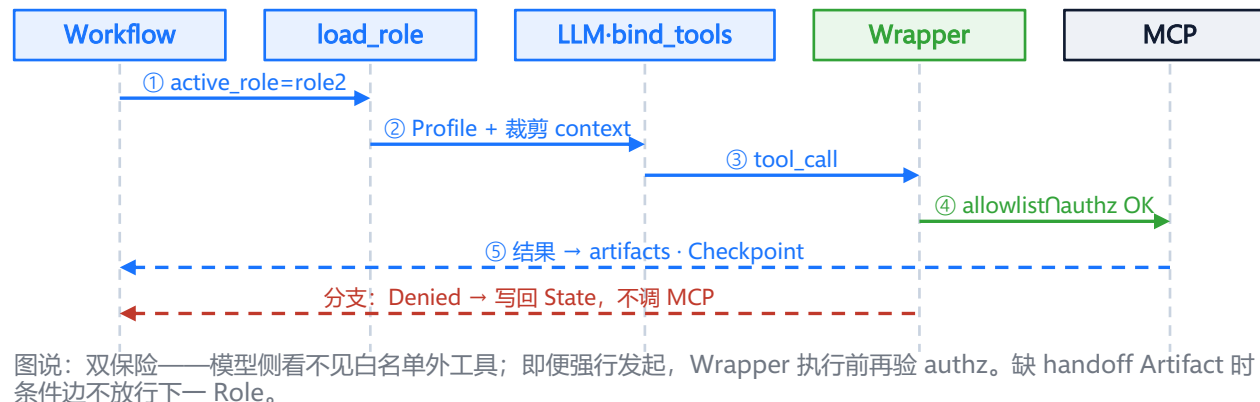
以「资产探查」为例：Profile → bind_tools + 上下文裁剪 + 鉴权

承接微案例 Role2 · Goal：客户日粒度风控特征表 · 探查节点不得创建生产任务或发布

```
# 本节点身份：资产探查（非建模 / 非发布）
role_id: role2_asset_discovery
# 可挂载的能力契约（编排层引用）
skills: [search_tables, infer_join_path, sample_profile, lineage_lookup]
# 模型可见工具面：故意不含写生产工具
tool_allowlist:
- meta.search_tables / get_columns / get_lineage # L0 只读
- query.sample_rows # L0 + PII 掩码
# 不含: job.create / release.publish # L2/L3
# 进模型前允许注入的知识切片（防上下文膨胀）
knowledge_view:
ontology_scopes: [customer, account, transaction]
metric_defs: [txn_cnt_30d, overdue_days, avg_txn_amt_30d]
# 执行前硬闸：环境 · 项目 · 行数 · PII
authz_scope:
envs: [dev, staging]
projects: [risk_cust_daily_feature]
row_limit: 100
pii_policy: mask
# 离开本节点前必须落盘：缺一则条件边不放行
handoff_contract:
required: [candidate_tables, field_mapping, evidence_refs, open_risks]
```

在 LangGraph 上如何实现

1. State: goal / active_role / role_profile / artifacts / messages
2. load_role: 读 Registry → 写入 state.role_profile
3. bind_tools(LangChain 模型 API): 仅 allowlist; 白名单外对模型不可见
4. context: 只注入 knowledge_view 允许切片
5. Tool wrapper: 执行前再验 authz_scope (双保险)
6. 条件边: 缺 handoff Artifact → 禁止进入 Role3



验收 1 · 越权拦截

探查节点若发起 release.publish（或白名单外工具）→ 模型侧不可见，或 Wrapper 返回 Denied，且不调 MCP。

对应：tool_allowlist + bind_tools + authz 二次闸

验收 2 · 交接守卫

artifacts 缺 field_mapping（或其它 required）→ 条件边禁止进入评估/建模（Role3），只能回探查或澄清。

对应：handoff_contract Schema 校验

验收 3 · 可审计回放

Checkpoint / Trace 能还原当时 active_role、工具白名单哈希、权限包版本——出事能回答「谁 · 看见什么 · 按什么权限」。

对应：role_profile 快照进 State + checkpointer

裁剪不是写在人设里，而是 Profile → bind_tools → context 组装 → 执行前 authz 的硬链路。

Role 的技术难点：边界、交接与拆分时机

对照第 15 页 Profile · 三个可复现难点收口 Role 段 · 验证标准：越权失败 / 交接可审 / 拆分可证

1 · 人设 ≠ 边界

Prompt 「你是探查专家」挡不住越权。

- 必须以第 12 页 allowlist + authz 为准
- 回归：白名单外工具调用必须失败
- 人设只能辅助表达，不能单独当安全边界

验收：发起 release.publish → Denied

2 · 共享 State vs 隔离

单图多 Role 默认共享 State；无 handoff 会污染下游。

- 对策：只交校验过的 Artifact，不是整段对话
- 缺 field_mapping 等 required → 条件边不放行
- 上游胡话不得直接进入 Role3

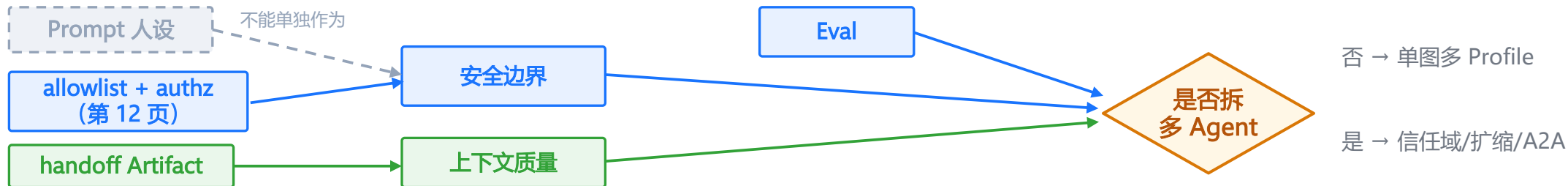
验收：缺交接物 → 不能进评估/建模

3 · 何时拆多 Agent

默认单图多 Role Profile（如第 12 页）。

- 再拆条件：独立信任域 / 独立扩缩 / 跨系统 A2A
- 合规要求进程级隔离时再拆
- 须有 Eval 证明拆分有收益，否则不拆

验收：拆分可证，而非「更像专家」



图说：人设不能单独当安全边界；真正边界来自第 12 页 allowlist + authz。handoff Artifact 决定下游上下文质量。是否拆多 Agent，由边界可靠、交接可审、Eval 可证三者共同决定。

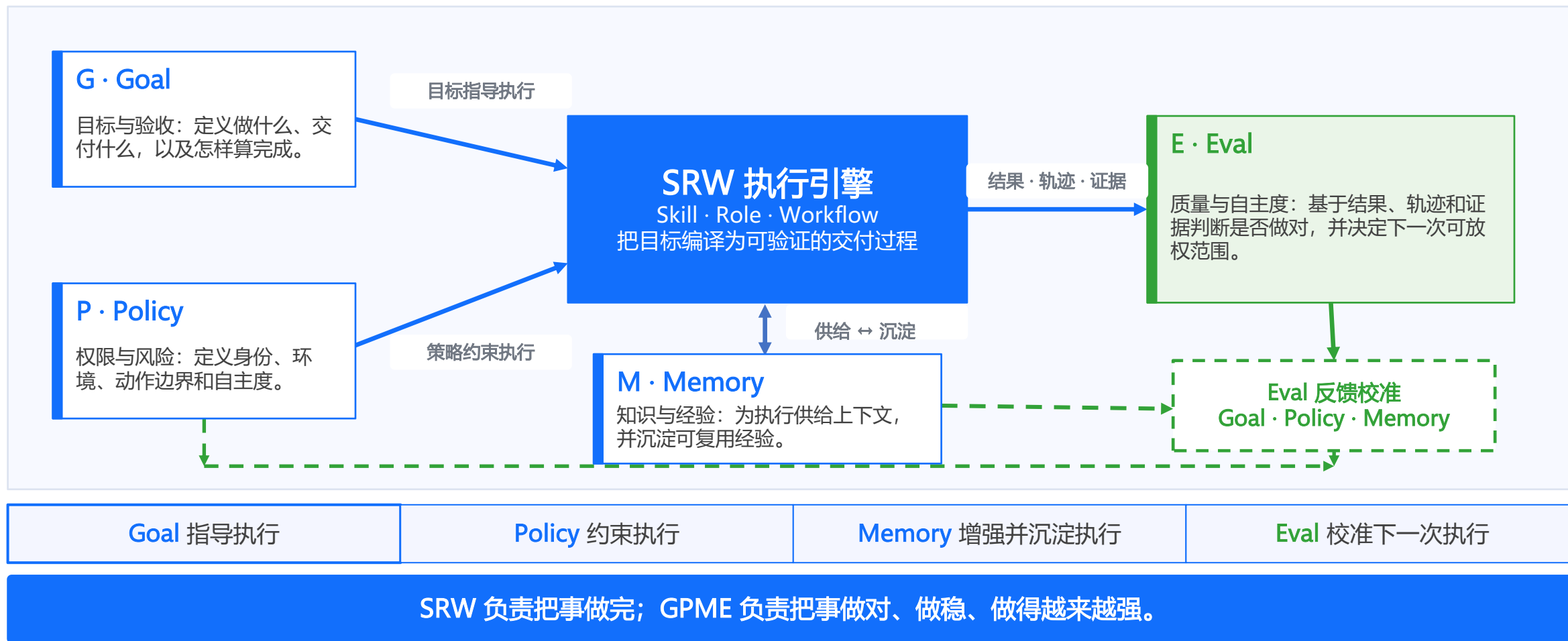
Role 验证完成标准：越权失败、交接可审、拆分可证——不是「对话更像专家」。

与后文双闻：Role 决定「看得见哪些工具」；Policy 决定「这一次能否自动做」。Harness 可补沙箱，但不替代 Profile。

03 GPME——从协同走向可委派自主

用目标、策略、记忆与评测换取可信自主

SRW 解决“如何执行”，GPME 解决“如何委派与进化”



人提交目标与边界，系统按风险决定自主度



Policy 的三条安全底线

风险分级决定“能否执行”，安全底线保证“执行时不越界”。

身份最小权限

用户、Agent、MCP Server 使用分离、短期且最小化的身份与凭证。

审批绑定动作

审批绑定主体、Tool、精确参数、目标资源、环境和有效期。

上下文全程不可信

校验 Tool、外部内容与 Memory，防工具投毒、提示注入和跨租户污染。

人工审批不是安全边界本身；如果审批没有绑定精确参数、资源和有效期，Agent 仍可能在审批后改变动作。

MEMORY × EVAL

Memory 让经验复利，Eval 让进化不漂移

Memory：作用域分层 × 四个正交维度

按作用域分层：决定知识可以在哪个范围复用

任务记忆

Goal、状态、中间产物

角色记忆

方法、纠错、偏好

团队记忆

规范、口径、实践

企业记忆

标准、资产、策略

按四个正交维度组织与检索：不是建设四套物理图谱

语义维度 · 是什么

同义、上下位、主题与指标口径

时间维度 · 何时发生

会话、版本、调度、时间窗与生命周期

关系 / 因果 · 为什么

依赖、影响、统计相关与有证据的因果链

实体维度 · 与谁有关

人、表、任务、指标与文档

写入前治理：来源与权威等级、敏感信息、租户隔离、版本 / TTL、Provenance 与完整性校验，防止 Memory / Context Poisoning。

Eval：证明做对，并决定下一次能否放权

Eval 不只判断最终答案，还要评估能力、运行质量和完整执行轨迹。

任务级 Goal 达成率、人工介入率、一次通过率、返工节点

能力级 找表 Top-K、口径准确率、规范违反率、无依据生成率

运行级 工具成功率与延迟、自动修复率、MTTR、Token 与成本

轨迹级 Context、Tool / 参数 / 顺序、Handoff、澄清恢复、权限与副作用

AgentOps 提供“可观察事实”，Eval 提供“质量判断”。

AgentOps 以 Trace / Span 记录 Agent、Model、Context、Tool、Guardrail、HITL 与 Handoff；Eval 消费这些数据形成评分、失败分类、回归集和发布门槛。

Eval + AgentOps 如何形成可审计的进化飞轮

1 · Goal + Context

带着目标、策略与当前知识进入执行

2 · AgentOps 采集

记录 Model、Tool、Handoff、HITL 完整 Trace

3 · Eval 评分

结果 / 语义校验 + 轨迹 / 安全评分 + 线上反馈

4 · 失败归因

定位能力、上下文、工具、策略或流程缺陷

5 · 形成改进

更新 Skill / Policy / Memory，并扩充 Dataset / Grader

6 · 人审与发布

版本化上线后进入下一轮执行与评测

生产 Trace → 可重复评测集 → 分阶段发布门槛；Prompt 与 Tool Result 默认仅记录摘要、哈希或脱敏内容。

自主度不是一次性开关，而是由 Goal 达成率、人工介入率和重大错误率等评测结果逐级解锁。

GPME · GOAL × POLICY IMPLEMENTATION

结构化 Goal + 调用前门闸：自主度做成系统对象

Spike · LangGraph 强类型 state + 工具调用前门闸 + interrupt / Checkpoint Resume · 非 DataCyber 生产承诺

Goal · 完成标准

结构化任务对象：交付物、验收、范围、环境、约束。写入 state.goal；入图前 Schema 校验。回答「完成长什么样」。

Policy · 自主度裁决

每次 tool_call 前的引擎（非 Prompt）。输入身份·等级·资源·环境·置信度 → allow / auto_validate / hitl / deny。

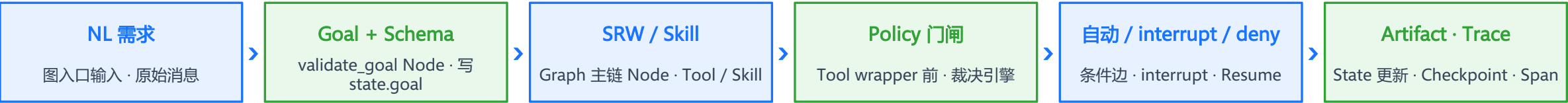
HITL · 高风险放行

L2/L3 或低置信度：interrupt + 绑参审批载荷（tool、args_hash、resource、env、有效期）。Resume 时重算 hash。

工具/动作等级 · L0→L3

工具调用的副作用等级（非 Agent 职级）。挂在工具元数据上。L0 只读 · L1 草稿 · L2 开发写入 · L3 生产变更 → 驱动 Policy 门闸。

Goal 字段	一句话
goal_id / scenario	任务身份与场景类型 → 路由主链 / 短路径
business_objective	业务目标一句话（人可读，供 Trace / 汇报）
deliverables[]	必须交付的产物清单 → 完成判定输入
acceptance[]	机器可检的完成标准 → Eval / Checker 对照
data_scope · env	主体/粒度与运行环境 → Policy 与 authz 输入
constraints · risk_flags	成本/时效/PII/合规 + 风险标记 → 门闸与 HITL 升级



Goal 回答「完成长什么样」； Policy 回答「这一步能不能自动做」——不能只写在系统提示里。



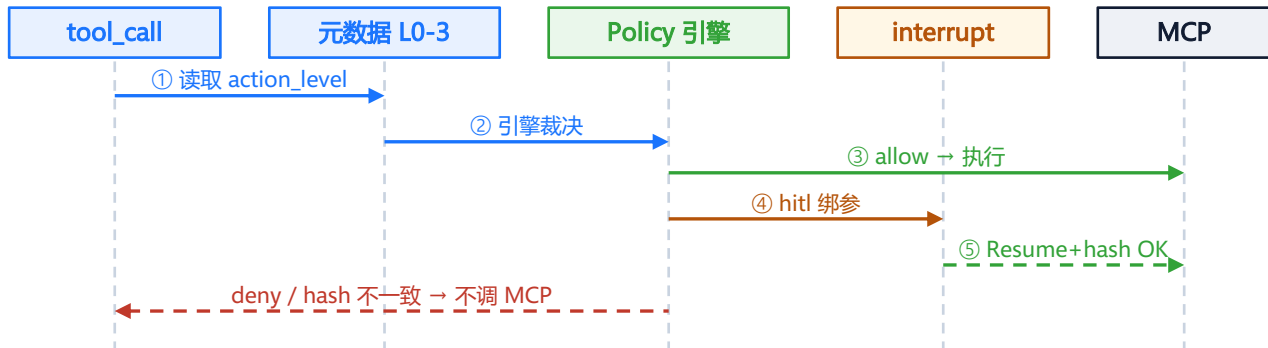
客户风控特征表 Goal; Policy 在 LangGraph 里如何拦 / 放

承接微案例 · Goal: 客户日粒度风控特征表 · Schema 拦带病开工 · Policy 拦带病放权

```
# 任务身份 · 场景路由 (主链 / 短路径)
"goal_id": "g_cust_risk_feature_001",
"scenario": "feature_table_build",
# 业务目标 (人可读 · Trace / 汇报)
"business_objective": "客户日粒度风控特征表 · 信贷审批",
# 交付物清单 → 完成判定输入
"deliverables": ["ddl", "etl_draft", "metric_dictionary", "test_report"],
# 机器可检验收 · 含指标与 PII 策略
"acceptance": [
  "txn_cnt_30d / overdue_days / avg_txn_amt_30d",
  "grain: customer_id + dt",
  "PII 按掩码策略处理"
],
# 范围 · 环境 → Policy / authz 输入
"data_scope": {"subjects": ["customer", "account", "transaction"]},
"env": "dev",
# 约束 + 风险标记 → 门闸升级 / HITL
"constraints": {"pii_policy": "mask", "regulatory": ["data_minimization"]},
"risk_flags": ["metric_cold_start_possible", "pii_sensitive"]
```

在 LangGraph 上如何实现

1. validate_goal: Schema → 写/冻 state.goal
2. Tool 元数据: 每工具挂 action_level L0-L3
3. policy_gate: ToolNode 前裁决引擎
4. allow / auto_validate: 执行 (后者先 Checker)
5. hitl: interrupt + args_hash 绑参
6. deny / Resume: 拒调 MCP · Checkpoint 续跑



图说: 门闸是工具路径上的同步裁决点; 审批绑定 args_hash, 防止「批了发布、执行时换参」。Goal 校验失败则根本不进本链路。

验收 1 · Schema 守卫

缺 acceptance / data_scope 等必填, 或 pii_sensitive 无掩码策略 → 进澄清节点, 禁止进入 SRW 主链。

对应: validate_goal Node · state.goal_errors

验收 2 · 绑参 HITL

L3 (或升级 hitl) 触发 interrupt; 人批后若改参导致 args_hash 变化 → 旧审批作废、拒绝执行。

对应: policy_gate · interrupt 载荷 · Resume 重算 hash

验收 3 · 双闸可审计

Role allowlist 让模型看不见发布工具; 即便误绑 L3, Policy 仍须 hitl/deny。Trace 可还原 policy_version 与裁决。

对应: 第 12 页 Role 闸 + 本页 Policy 闸 · Checkpoint/Span

Goal 靠 Schema 拦住「带病开工」; Policy 靠调用前门闸拦住「带病放权」。

GP 的技术难点：分级、绑参、裁决归属与续跑

对照第 22 页门闸 · 四个可复现难点收口 GP 段 · 验证标准：高风险拦住 / 绑参失效 / 策略可回归 / 中断可续跑

1 · 动作分级难于写文案

工具元数据漏标/错标 = 放权错误。

- L0-L3 挂在工具元数据，非 Prompt 形容词
- PoC 先覆盖主链高频 Tool，再扩长尾
- 漏标生产写入工具 → 假自动

验收：主链 Tool 全有 action_level

2 · 审批未绑参 = 假安全

批了发布、执行时换参，等于没批。

- 第 18 页 args_hash 是验收项，不是细节
- Resume 必须重算 hash；不一致则拒绝
- 载荷含 tool / args / resource / env / 有效期

验收：改参后旧审批必须失效

3 · 裁决不交给 LLM 临场发挥

LLM 可建议；引擎裁决才能回归。

- Policy 是纯函数/规则引擎，写入 Trace
- 记录 policy_version、输入指纹、裁决结果
- 同输入必须同裁决，否则无法评测放权

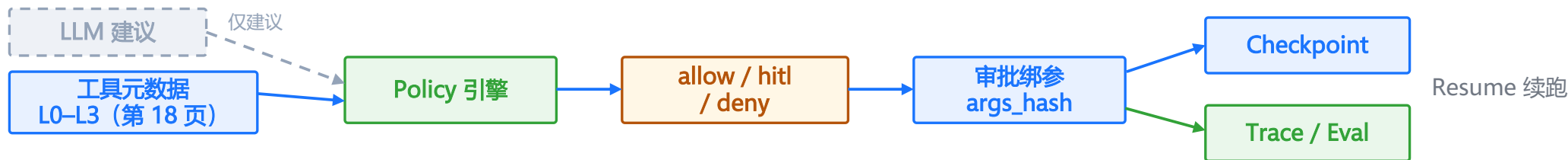
验收：同指纹复现同一裁决

4 · Resume 而非整链重来

HITL 后应从 Checkpoint 续跑当前工具步。

- 整链重跑破坏 Goal 状态机与 Eval 归因
- Durable：中断前后状态可审计
- 与第 18 页 interrupt / Resume 链路一致

验收：审批后续跑未完成步，非整链



图说：真正定级来自工具元数据；LLM 只能建议。引擎产出 allow/hitl/deny（图为简化，省略 auto_validate 分支）后，HITL 必须绑参 hash，再经 Checkpoint Resume，最后进入 Trace/Eval——缺一环就会变成假安全或假自动。

委派验证完成标准：高风险默认拦住、审批不能偷换参数、策略可回归、中断后可续跑。

04 实践价值与落地路径

以平台能力为底座，按风险与证据逐级放权

先在低风险、高频、可评测场景建立信任，再逐步扩大自主范围

当前边界：为什么不能一步到“全自主”

这些是企业 Data Agent 落地时必须显式管理的能力上限与风险条件。

1· 语义冷启动

缺少权威口径时，Agent 必须降级、引用证据或向人澄清。

2· 生产写入风险

删除、发布、生产重跑等难回滚动作仍需要强门禁。

3· 跨域泛化有限

一个业务域积累的经验，不能直接升级为另一业务域的事实。

4· 端到端评测不足

SQL 可运行不等于业务口径正确，自动 Grader 仍有盲区。

5· 企业数据复杂性

跨数据库、脏 Join Key、非结构化转换与领域知识仍是难点。

6· 性能与成本约束

资产检索、长链路和多模型调用需要并行化与模型路由。

建议落地路线：工程能力、场景风险与自主度同步升级

P0-P4 是成熟度阶段，不是简单的产品版本号；每一步都必须由 Eval 结果解锁。

P0· 打基础

语义契约·工具标准化
结构化 Goal + Policy



P1· 跑通主链

SRW 执行闭环
Context + Durable State



P2· 建立信任

Trace + 轨迹评测
只读场景高自主



P3· 经验回流

Memory + 语义回流
开发写入条件自主



P4· 审慎扩展

跨域复制
生产动作逐级解锁

组织要同步调整：工程师从执行者转向目标定义者和例外决策者。

人的重点工作转向验收标准、风险规则、证据审查与高影响例外处置。

外部基准参考 | Text-to-SQL 不等于 Data Agent: 2026 年 Data Agent Benchmark (DAB) 覆盖 12 个数据集、54 个问题、9 个领域和 4 类 DBMS，当时最佳前沿模型约为 38% Pass@1。该数据不是 DataCyber 实测，也不用于比较模型；它只提醒我们，真实 Data Agent 的难点已从“生成一条 SQL”转向跨系统整合、语义判断、证据验证和长链任务。

从“Agent 会干活”到“企业敢委派”



05 DataCyber的Agent Ready能力

语义智能 + 协议化工具面 + 可信自治

DATA CYBER · AI-NATIVE REFERENCE ARCHITECTURE

语义智能 + 协议化工具面 + 可信自治: DataCyber AI-Native 技术栈



语义智能+协议化工具面+可信自治: DataCyber AI-Native 技术栈

Data Agent Ready ≠ 增加聊天入口; 它要求平台能力可连接、语义可理解、动作可执行、过程可治理。

Semantic-first

本体与语义契约先于生成

Protocol-first

Tool Schema + MCP 开放工具面

Durable & Governed

State + Policy + HITL 可恢复执行

Eval-driven

Trace → Grader → 发布门槛

数据与生产底座

平台现有

- 多云、多引擎、湖仓与流批资源
- 集成、ETL、调度、任务与实例
- 元数据、资产、血缘、指标与质量
- 监报告警、数据源、API 与数据服务

MCP / A2A 与开放连接

接入能力

- MCP External Client 作为外部入口
- 执行层 MCP Client 调用外部能力
- 资产、任务、集成、质量、工作流、API / AI 等 Tool 域
- 产品目标: 经 Policy Gate 受控 MCP Server 化
- A2A 仅承接跨系统、独立信任边界的 Agent 委派

入口、编排与运行

平台现有

- Graph SSE、SmartQuery、SQL Review
- Coordinator 意图识别与任务分解
- Agent 路由、并行 / 串行和结果聚合
- StateGraph 节点编排、模型注册与 Tool Calling

语义、知识与 Context

平台现有

- Ontology、术语解析、概念映射
- 语义搜索、表发现与 JOIN 路径推理
- 本体增强 NL2SQL 与 RAG 问答
- MySQL + Neo4j + Elasticsearch 协同

平台 Data Agent

Agent 能力

- DEA: 探查、血缘、关系与智能问数
- POA: 同步、ETL、调度与任务监控
- PQA: 元数据、口径、规范与运维问答
- DQA / COA: 质量闭环与复杂任务协调

SRW + GPME + AgentOps

工程增强

- 语义契约 / Tool Schema、职责 Role、Durable Workflow
- Goal 验收、Policy 风险、Memory 治理
- Trace、轨迹 Eval、HITL 与版本发布
- 候选 Skill / 知识经人审和回归后生效

状态说明: “平台现有 / 接入能力” 表示当前可复用的数据、运行、语义与连接能力; “工程增强” 是产品化升级方向, 正式宣传前需确认上线状态。

业务增强优先, 平台原生兜底:

DataCyber Data Agent Ready 的核心, 是让已有数据、语义和生产能力通过标准接口安全调用, 再由 SRW + GPME 组织成可委派闭环。



术语表: Data Agent 工程概念速查

补充页 · 不占主演讲时间 | 用于统一产品、架构与工程团队的讨论语言

A2A	独立 Agent 间的能力发现、任务委派、状态与 Artifact 交换协议。	HITL	在信息不足、关键决策或高风险动作中引入人工确认。
AgentOps	对 Model、Context、Tool、Guardrail、成本、轨迹与结果追踪运营。	MCP	连接 Agent 与 Tool / Data / Resource / Context 的标准协议。
Artifact	可版本、可验证、可审计的交付物，如 SQL、报告、证据与审批记录。	Memory	经来源、权限、版本、TTL 和完整性治理后可复用的知识。
Context Engineering	逐步选择、裁剪、隔离和压缩上下文，形成最小高信号输入。	Policy	按身份、环境、风险和证据决定工具可见性、权限与 HITL。
Data Agent	理解数据上下文、调用数据工具并完成分析或数据工程任务的智能体。	Role	绑定 Skill、知识视角、权限与协作边界的职责抽象。
Data Agent Ready	平台能力可被 Agent 标准理解、访问、执行、追踪与治理。	Semantic Contract	对实体、指标、粒度、Join 与时间语义的可执行、版本化定义。
Durable Execution	以持久状态、Checkpoint、幂等、Resume 与补偿保障长任务。	Skill	带输入输出契约、工具、规范、校验与评测的原子能力。
Eval	评估结果、能力、运行和轨迹，并据此决定发布与放权。	SRW	Skill + Role + Workflow：把能力、责任和交付闭环工程化。
Goal	包含目标、交付物、验收标准与约束的结构化任务对象。	Workflow	管理状态、节点、分支、恢复、补偿与人工门禁的执行流程。
GPME	Goal + Policy + Memory + Eval：可委派自主的治理控制面。	数据智能中枢	为 Agent 提供数据、语义、工具、治理和反馈闭环的运行体系。

SRW + GPME 是本次演讲提出的工程抽象，不应表述为行业既有标准。

Q6 Q&A

从哪个数据场景开始,
你最愿意把目标交给 Agent?

语义层冷启动

MCP 工具粒度

HITL 确认点

端到端评测

多云权限统一

SRW + GPME 与 DataOps

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会